

FIAP · PÓS-TECH · SOAT

Documentação de Arquitetura

Tech Challenge — Fase 3

Sistema de Gestão de Oficina Mecânica

ADRs · RFCs · Diagramas de Componentes e Sequência · Modelo de Dados

Organização GitHub: **fiap-officine**

Nuvem de referência: **AWS** (EKS · API Gateway · Lambda · RDS PostgreSQL)

Grupo 123

Documentação de Arquitetura — Oficina Mecânica (Fase 3)

Este diretório reúne a documentação arquitetural da aplicação de gestão de oficina mecânica, evoluída na Fase 3 para operação em nuvem com autenticação serverless, banco gerenciado, cluster Kubernetes escalável, observabilidade e CI/CD segregado por repositório.

Nuvem de referência: AWS. A justificativa formal está na [RFC-001](#). Todo o desenho se traduz de forma equivalente para GCP/Azure (API Gateway → Apigee/APIM, Lambda → Cloud Functions/Azure Functions, RDS → Cloud SQL/Azure Database, EKS → GKE/AKS).

Índice

Documento	Conteúdo
Modelo de Dados	Justificativa do banco, diagrama ER e relacionamentos
ADRs	Decisões arquiteturais permanentes
RFCs	Decisões técnicas em discussão/proposta

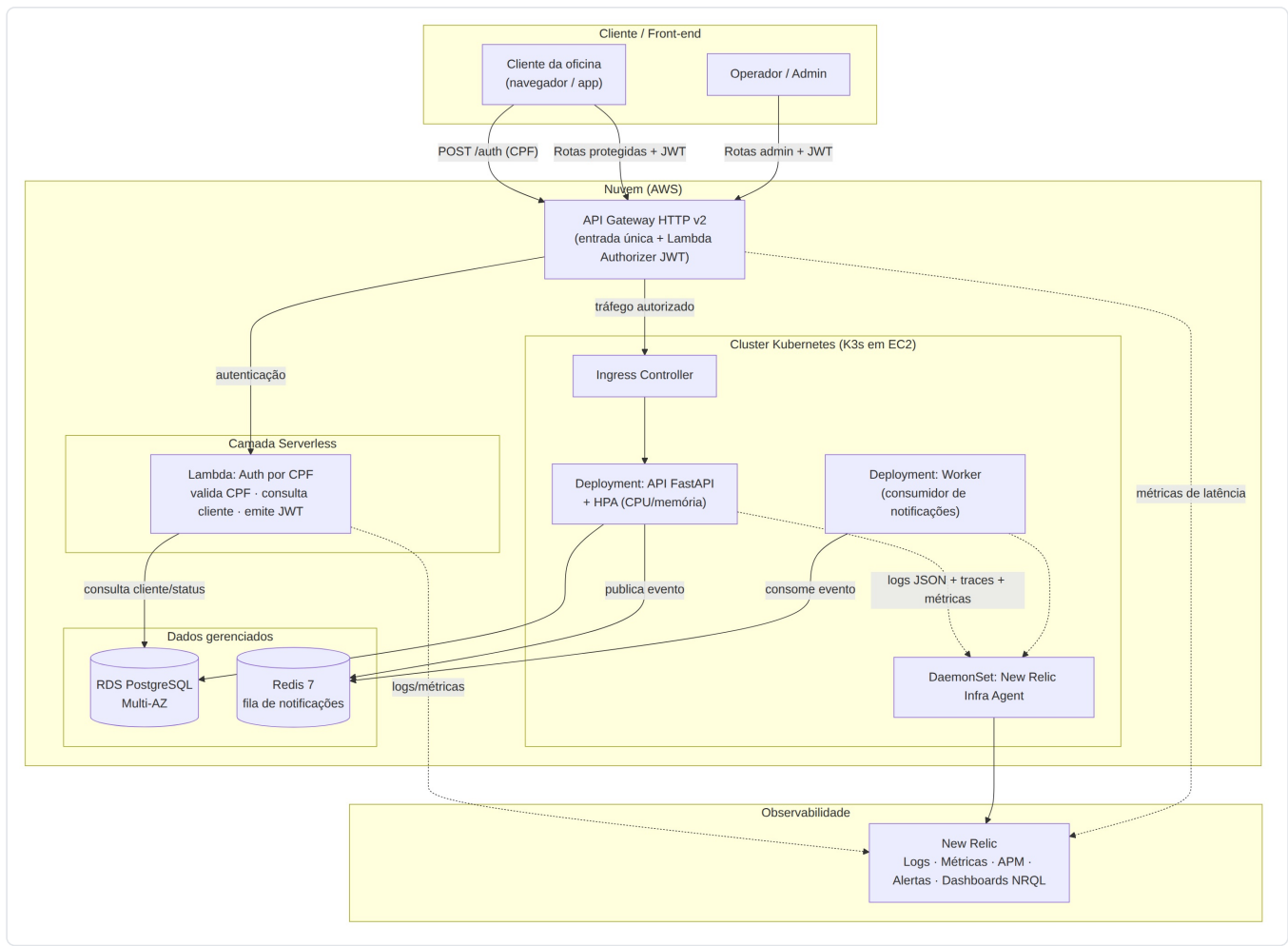
1. Visão geral

A aplicação é um sistema de gestão de ordens de serviço (OS) para uma rede de oficinas com múltiplas unidades. O núcleo de negócio (API + worker) roda em Kubernetes; a autenticação de clientes por CPF é feita de forma **serverless** (fora do cluster) e todo o tráfego externo entra por um **API Gateway** único.

Componentes principais:

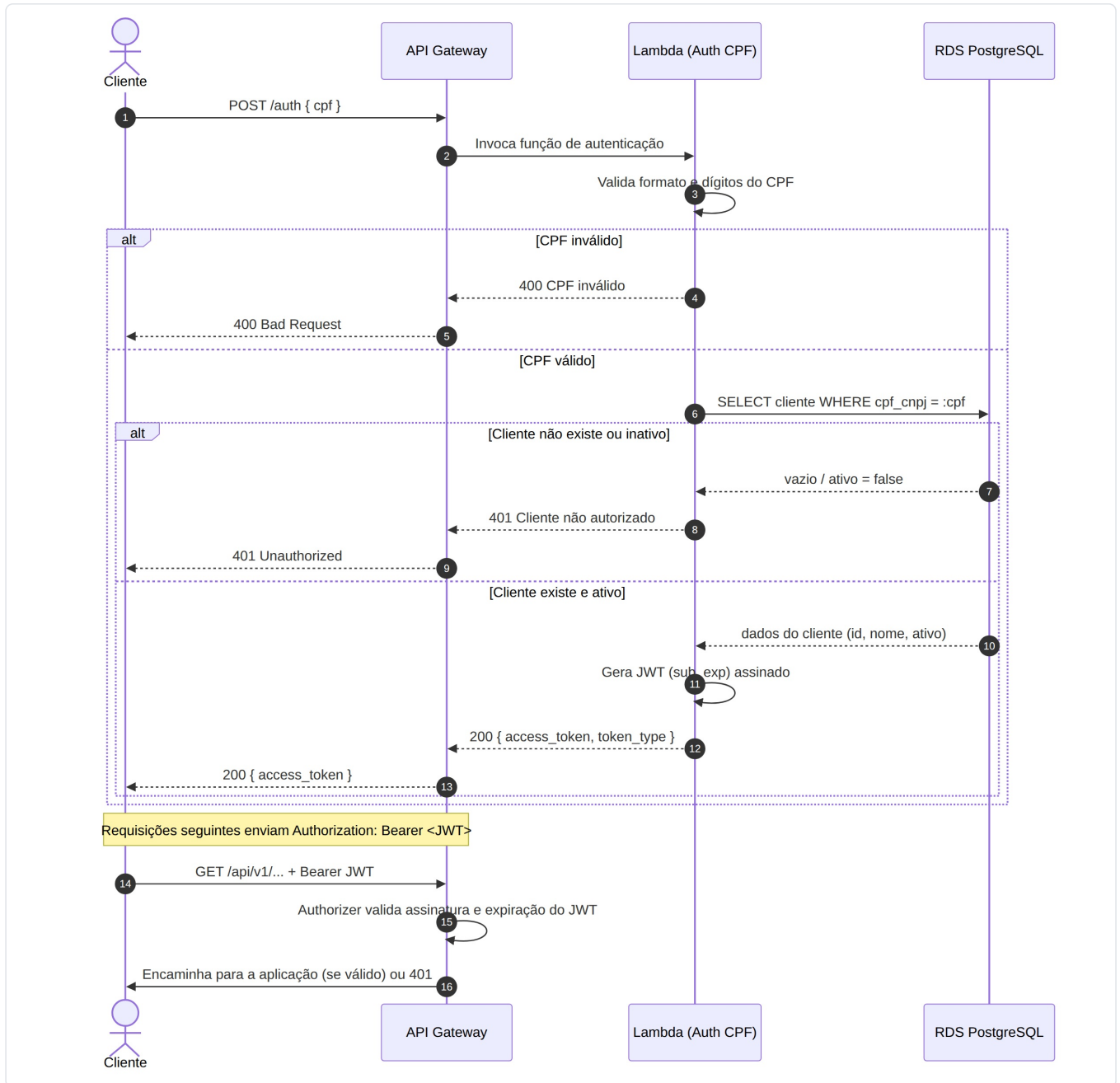
- **API Gateway** — ponto único de entrada, roteamento e proteção das rotas sensíveis (ver [ADR-003](#)).
- **Função Serverless de Autenticação (Lambda)** — valida o CPF, consulta a existência/status do cliente no banco e emite um JWT.
- **Aplicação principal (FastAPI)** — regras de negócio de OS, clientes, veículos, serviços e peças; roda em Kubernetes com HPA.
- **Worker de notificações** — consome eventos da fila (Redis) e dispara notificações de forma assíncrona.
- **Banco de dados gerenciado (RDS PostgreSQL)** — persistência transacional (ver [RFC-002](#) e [ADR-005](#)).
- **Observabilidade (New Relic)** — logs estruturados, métricas, APM e alertas (ver [RFC-004](#)).

2. Diagrama de Componentes (visão de nuvem)



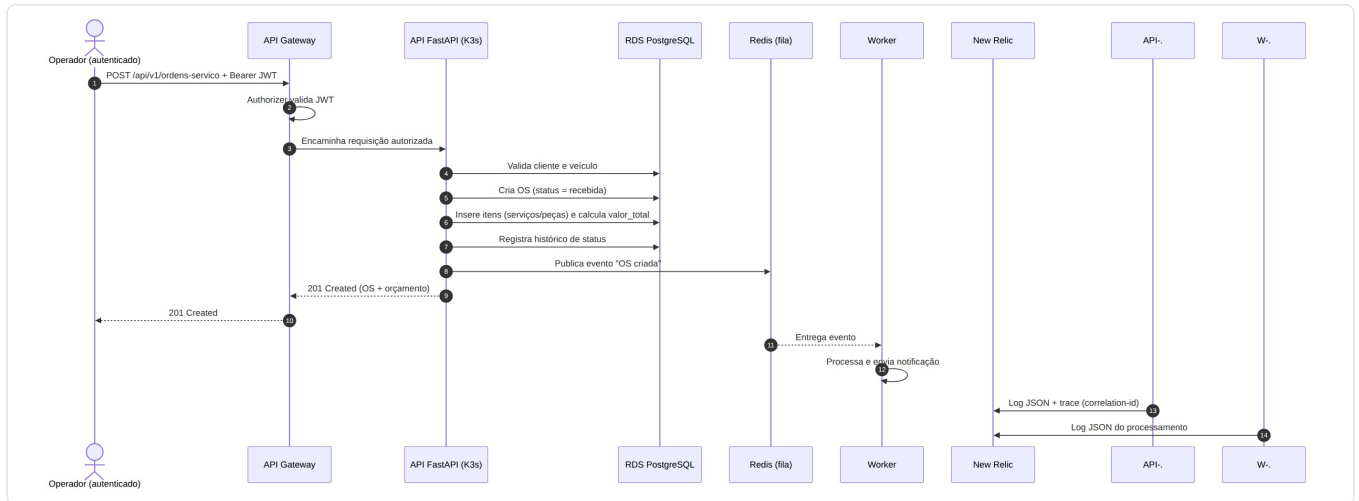
3. Diagrama de Sequência — Autenticação por CPF

Fluxo serverless de autenticação. O cliente informa o CPF; a função valida o documento, confirma a existência e o status na base e devolve um JWT usado nas rotas protegidas.



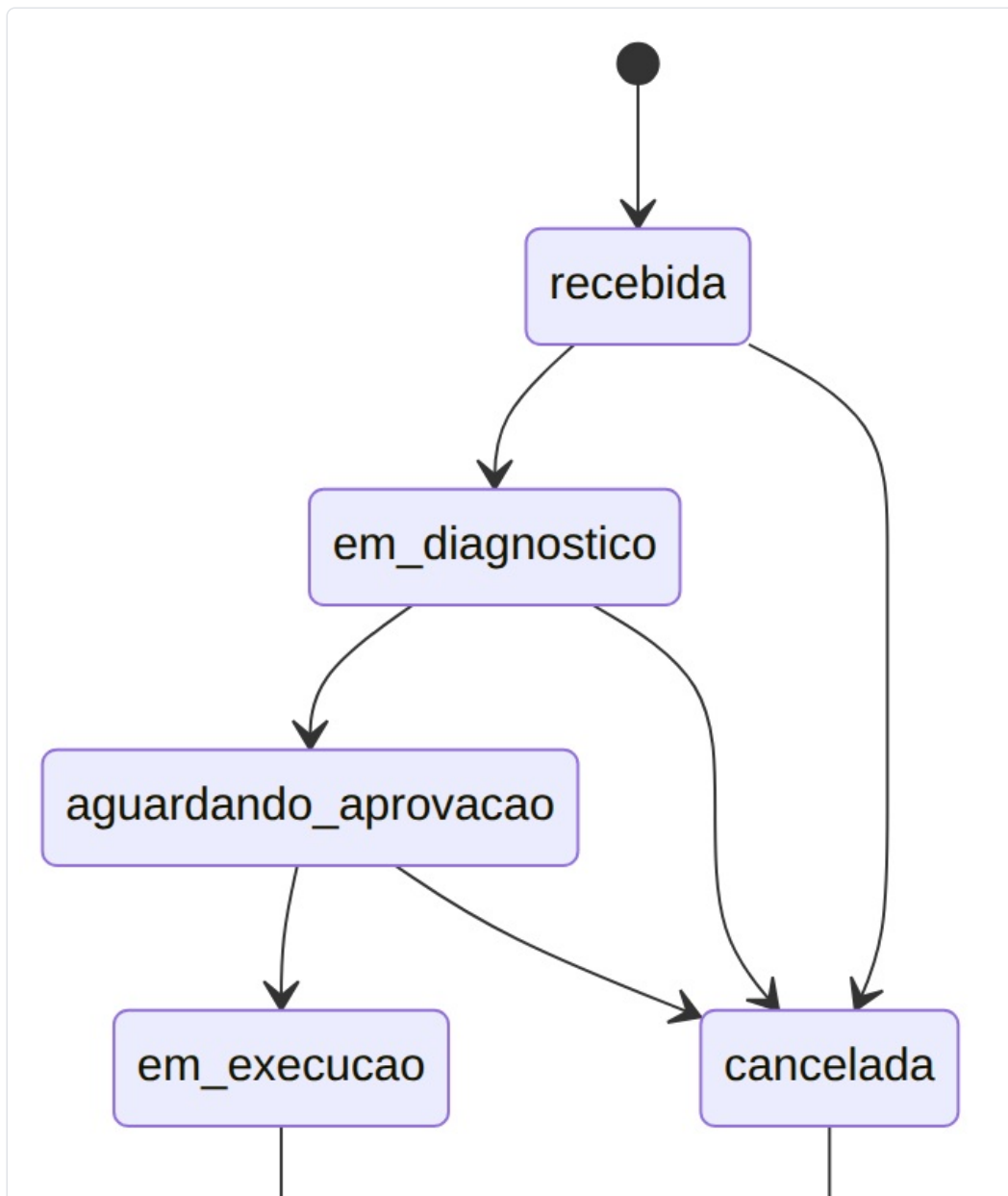
4. Diagrama de Sequência — Abertura de Ordem de Serviço

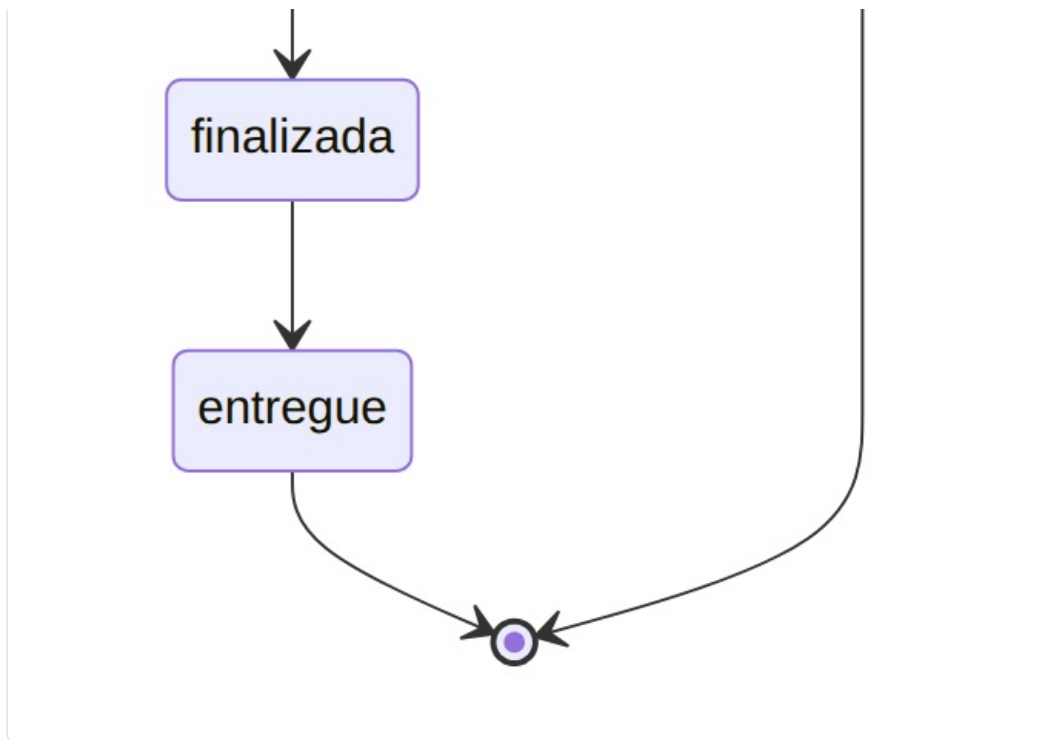
Fluxo de negócio de criação de uma OS já autenticado, incluindo cálculo automático de orçamento e notificação assíncrona via fila.



5. Fluxo de status da OS

Máquina de estados implementada em `app/domain/enums.py` (`TRANSICOES_VALIDAS`). Transições fora deste grafo são rejeitadas pela camada de serviço.





6. Rastreabilidade requisito → documento

Requisito do enunciado (Fase 3)	Onde é atendido
API Gateway para roteamento/controle	ADR-003 , §2
Function serverless de autenticação por CPF	RFC-003 , §3
Banco de dados gerenciado	RFC-002 , ADR-005
Cluster Kubernetes escalável (HPA)	ADR-002
Terraform / IaC e 4 repositórios	ADR-004
Monitoramento e observabilidade	RFC-004
Padrão de comunicação	ADR-001
Diagramas de componentes e sequência	Este documento, §2-§4
Justificativa do banco + modelo ER	data-model.md

Modelo de Dados — Justificativa e Diagrama ER

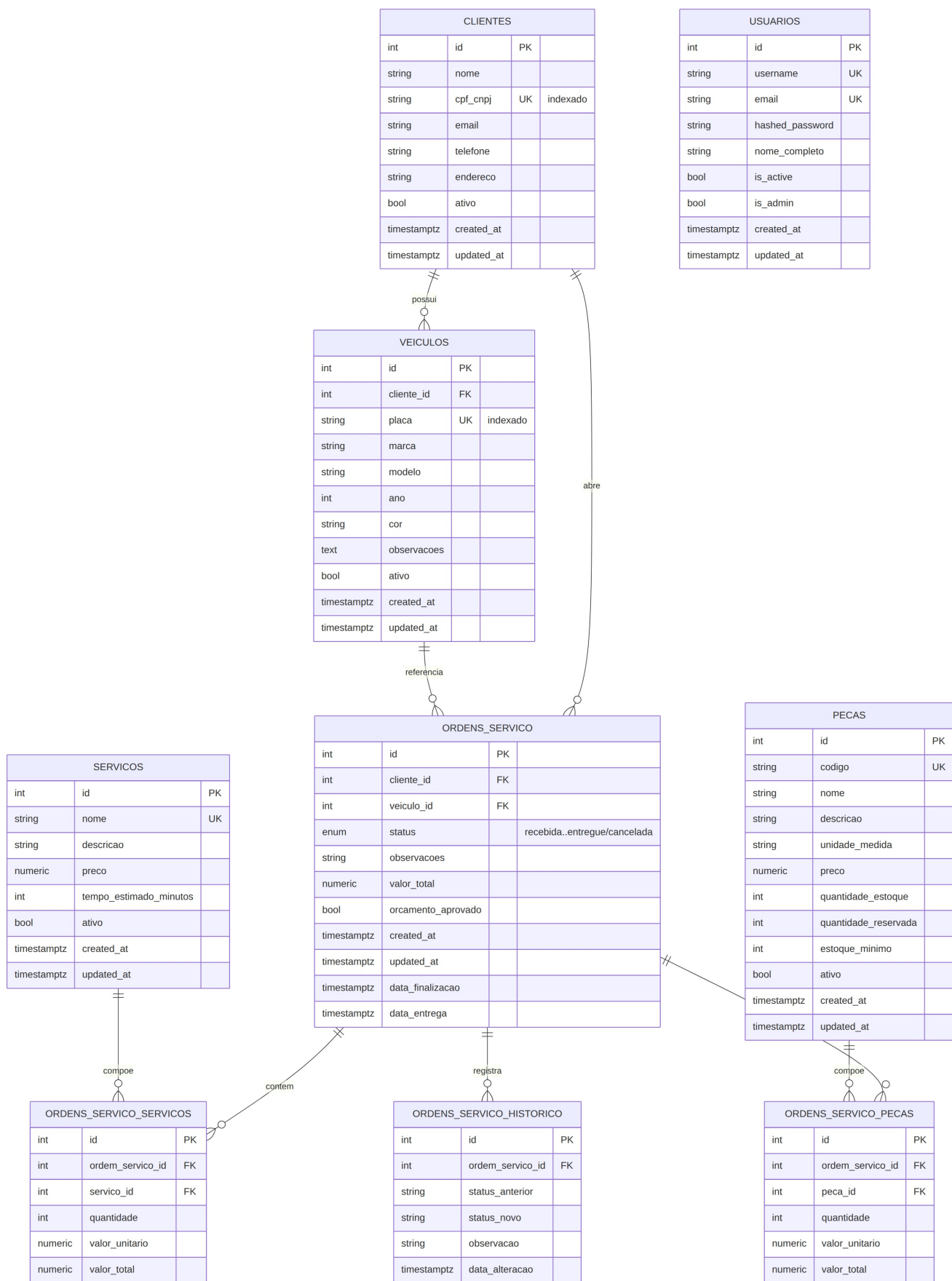
1. Justificativa formal da escolha do banco de dados

O banco escolhido é o **PostgreSQL** (na Fase 3, na modalidade **gerenciada — Amazon RDS**). A decisão está formalizada na [RFC-002](#) e na [ADR-005](#). Resumo dos motivos:

- **Conformidade ACID.** O domínio manipula estoque de peças, valores de orçamento e transições de status de OS. Essas operações exigem atomicidade e consistência forte — um banco relacional transacional é o ajuste natural, e um NoSQL exigiria reconstruir garantias que o PostgreSQL já oferece de fábrica.
- **Relacionamentos ricos.** O modelo é fortemente relacional: cliente → veículos → ordens de serviço → itens (serviços/peças). Consultas de negócio (histórico, tempo médio por status, OS por cliente) dependem de JOINS, chaves estrangeiras e integridade referencial.
- **Tipos adequados ao domínio.** `NUMERIC(10,2)` para valores monetários (sem erro de ponto flutuante), `ENUM` nativo para o status da OS e `TIMESTAMPZ` para datas com fuso.
- **Ecossistema.** Integração madura com SQLAlchemy 2.0 (ORM) e Alembic (migrations), já em uso no projeto.
- **Operação gerenciada.** Em RDS, o time deixa de operar backup, patching, failover e réplicas manualmente. Multi-AZ entrega alta disponibilidade, um dos objetivos explícitos da fase.

2. Diagrama Entidade-Relacionamento

Modelo derivado das entidades em `app/domain/entities/`.



USUARIOS é uma entidade isolada (operadores/admin do back-office) e não se relaciona com o domínio de OS — daí aparecer sem arestas no diagrama.

3. Explicação dos relacionamentos

- **Cliente 1—N Veículo** (`veiculos.cliente_id` → `clientes.id`). Um cliente pode ter vários veículos; todo veículo pertence a exatamente um cliente.
- **Cliente 1—N Ordem de Serviço** (`ordens_servico.cliente_id` → `clientes.id`). A OS guarda o cliente de forma direta (além do vínculo pelo veículo) para simplificar consultas de "OS por cliente" e o endpoint público de acompanhamento por CPF.
- **Veículo 1—N Ordem de Serviço** (`ordens_servico.veiculo_id` → `veiculos.id`). Cada OS trata de um veículo específico; o veículo acumula seu histórico de OS.
- **Ordem de Serviço N—N Serviço**, resolvida pela tabela associativa `ordens_servico_servicos` . A associativa carrega `quantidade` , `valor_unitario` e `valor_total` — o preço é **congelado no momento da inclusão**, para que reajustes futuros no catálogo de serviços não alterem orçamentos já emitidos.
- **Ordem de Serviço N—N Peça**, resolvida por `ordens_servico_pecas` , com a mesma estratégia de congelar preço e quantidade. É aqui que o consumo de peças se conecta ao controle de estoque (`pecas.quantidade_estoque` / `quantidade_reservada`).
- **Ordem de Serviço 1—N Histórico** (`ordens_servico_historico`). Cada transição de status gera um registro (`status_anterior` , `status_novo` , `data_alteracao`), servindo de trilha de auditoria e base para métricas de tempo por status.

O `valor_total` da OS é a soma dos `valor_total` das associativas de serviços e peças (orçamento automático).

4. Ajustes no modelo relacional para a Fase 3

Evoluções recomendadas ao promover o banco a gerenciado, sem quebrar o modelo:

1. **Índices de suporte às consultas de negócio e observabilidade** - `ordens_servico(status)` — já indexado; sustenta a listagem por status e o dashboard de volume por status. - Índice composto `ordens_servico(status, created_at)` — acelera o "volume diário por status" exibido no dashboard (RFC-004). - `ordens_servico_historico(ordem_servico_id, data_alteracao)` — cálculo de tempo médio por status.
2. **Consistência de estoque.** Reserva/baixa de `quantidade_estoque` e `quantidade_reservada` deve ocorrer dentro da mesma transação da inclusão de peças na OS, com `SELECT ... FOR UPDATE` na linha da peça para evitar corrida entre unidades concorrentes.
3. **Chave de correlação.** Coluna opcional `correlation_id` (ou uso do `id` + trace do New Relic) para amarrar logs/traces a uma OS específica.
4. **Migrations versionadas.** Todas as alterações continuam via Alembic (`alembic/versions/`), aplicadas pelo pipeline do repositório de banco antes do deploy da aplicação.
5. **Segurança gerenciada.** Credenciais do RDS via Secrets Manager; conexão TLS obrigatória; instância em subnet privada, acessível apenas pelo cluster e pela Lambda de autenticação.

Architecture Decision Records (ADRs)

ADRs registram **decisões arquiteturais permanentes** — escolhas estruturais que moldam o sistema e cujo custo de reversão é alto. Cada ADR segue o formato: Contexto, Decisão, Consequências e Alternativas consideradas.

Diferença para RFCs: a **RFC** propõe e discute uma decisão técnica (ex.: qual nuvem, qual banco); o **ADR** consolida a decisão arquitetural e suas implicações estruturais (ex.: padrão de comunicação, uso de HPA). RFCs desta pasta vizinha alimentam os ADRs correspondentes.

ADR	Título	Status
ADR-001	Padrão de comunicação entre componentes	Aceito
ADR-002	Uso de HPA para escalabilidade horizontal	Aceito
ADR-003	API Gateway como entrada única e autenticação serverless	Aceito
ADR-004	Segregação em quatro repositórios com CI/CD independente	Aceito
ADR-005	Banco de dados gerenciado desacoplado do cluster	Aceito

Convenção de status

Proposto → Aceito → (Substituído por ADR-XXX | Descontinuado)

ADR-001 — Padrão de comunicação entre componentes

- **Status:** Aceito
- **Data:** 2025-07
- **Decisores:** Grupo 123
- **Relacionado a:** [ADR-003](#), [RFC-004](#)

Contexto

O sistema tem dois tipos de interação com naturezas distintas:

1. **Requisições de negócio** (criar OS, consultar cliente, alterar status) — exigem resposta imediata e síncrona ao chamador.
2. **Efeitos colaterais** (notificar cliente sobre criação/alteração de OS) — não precisam bloquear a resposta e podem falhar/reprocessar sem impactar a operação principal.

Tratar os dois casos com o mesmo mecanismo síncrono acoplaria a latência da API à disponibilidade do canal de notificação, prejudicando o objetivo de alta disponibilidade da fase.

Decisão

Adotar um **padrão híbrido**:

- **REST/HTTP síncrono** para toda a comunicação externa (cliente ↔ API Gateway ↔ aplicação). Contrato descrito em OpenAPI ([docs/openapi.json](#)), verbos e códigos HTTP semânticos.
- **Mensageria assíncrona via fila (Redis)** para notificações. A API publica um evento ao criar/alterar uma OS; um **worker** desacoplado consome a fila e executa a notificação.

A autenticação por CPF é síncrona, mas isolada em função serverless (ver ADR-003).

Consequências

Positivas - Latência da API independente do canal de notificação. - Worker pode ser escalado e reiniciado sem afetar a API. - Picos de notificação são absorvidos pela fila (buffer natural).

Negativas / trade-offs - Entrega assíncrona é *eventual*: a notificação não é instantânea. - Introduz um componente de infraestrutura a operar (Redis) e a necessidade de tratar reprocessamento e mensagens mortas. - Exige correlação de logs (correlation-id) para rastrear uma operação que atravessa API e worker — endereçado na RFC-004.

Alternativas consideradas

- **Tudo síncrono (chamada direta de notificação na request)**: rejeitado por acoplar disponibilidade e latência.
- **Broker mais robusto (SQS/RabbitMQ/Kafka)**: adequado para escala maior, mas o Redis já está no stack e atende ao volume atual; a migração é possível sem mudar o padrão (apenas o transporte). Registrado como evolução futura.

ADR-002 — Uso de HPA para escalabilidade horizontal

- **Status:** Aceito
- **Data:** 2025-07
- **Decisores:** Grupo 123
- **Relacionado a:** [ADR-005](#), [RFC-004](#)

Contexto

Com a expansão para múltiplas unidades, a carga sobre a API é variável: picos em horário comercial, vales à noite. Manter um número fixo de réplicas ou significa desperdício de recurso (superprovisionamento) ou risco de degradação sob pico (subprovisionamento). A fase exige um cluster Kubernetes (K3s em EC2) **com escalabilidade**.

Decisão

Usar o **Horizontal Pod Autoscaler (HPA)** do Kubernetes para a API, escalonando o número de réplicas com base em **utilização de CPU e memória**. O manifesto vive em `k8s/api-hpa.yaml`.

Parâmetros de referência: - `minReplicas: 2` (garante disponibilidade mesmo em carga baixa e permite rolling update sem downtime). - `maxReplicas: 10` (teto para conter custo). - Alvo de **70% de CPU** e **75% de memória**.

O **worker** de notificações também é escalável horizontalmente (múltiplos consumidores da fila), mas com política própria por profundidade de fila, independente do HPA da API.

Consequências

Positivas - Ajuste automático à demanda, atendendo disponibilidade e custo. - `minReplicas ≥ 2` remove ponto único de falha na API. - Integra-se ao objetivo de observabilidade: as mesmas métricas de CPU/memória monitoradas no New Relic alimentam a decisão de escala.

Negativas / trade-offs - Exige `resources.requests/limits` bem calibrados em cada container — sem eles o HPA não funciona corretamente. - Escala reativa: há um atraso entre o pico e a subida de réplicas. Mitigável com `behavior` de scale-up mais agressivo se necessário. - O banco precisa suportar o aumento de conexões concorrentes das novas réplicas (ver ADR-005 — pool de conexões e limites do RDS).

Alternativas consideradas

- **Escala vertical (aumentar recursos do pod):** limitada pelo tamanho do nó e não elimina ponto único de falha.
- **Réplicas fixas:** simples, porém ineficiente e arriscado sob picos.
- **KEDA (escala por eventos):** interessante para o worker (escala por profundidade de fila); registrado como evolução, mantendo HPA para a API.

ADR-003 — API Gateway como entrada única e autenticação serverless

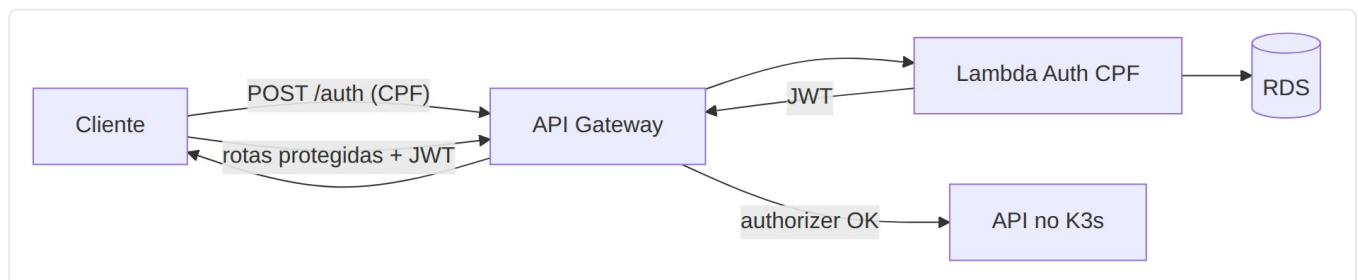
- **Status:** Aceito
- **Data:** 2025-07
- **Decisores:** Grupo 123
- **Relacionado a:** [RFC-003](#), [RFC-001](#)

Contexto

A Fase 3 exige controlar acessos e autenticações com segurança, proteger rotas sensíveis com autenticação via CPF e adotar soluções **serverless** para autenticação. Na Fase 2 a autenticação era feita dentro do próprio monólito (usuário/senha + JWT), o que acopla o ciclo de vida do mecanismo de auth ao da aplicação e não oferece um ponto único de controle de tráfego externo.

Decisão

1. **API Gateway (HTTP API v2) como ponto único de entrada.** Todo o tráfego externo passa pelo gateway, responsável por roteamento, throttling, TLS e validação do JWT via **Lambda Authorizer** antes de encaminhar às rotas protegidas no cluster.
2. **Autenticação por CPF em função serverless (Lambda), fora do cluster.** A função: - valida o formato e os dígitos verificadores do CPF; - consulta a existência e o **status** (**ativo**) do cliente no banco; - gera e devolve um **JWT** assinado, válido para consumir as APIs protegidas.
3. A aplicação em Kubernetes deixa de emitir token para clientes; confia no JWT validado pelo gateway. O login administrativo (operadores) permanece como fluxo próprio.



Consequências

Positivas - Autenticação desacoplada, escalando de forma independente e com custo por uso (serverless, sem pod ocioso).
- Ponto único para aplicar segurança (throttling, WAF, rate limit) e observar latência de borda.
- Superfície de ataque reduzida: a aplicação só recebe tráfego já autorizado.

Negativas / trade-offs - Mais peças de infraestrutura (gateway + função) e uma dependência de rede da Lambda ao banco (exige VPC/subnet configurada).
- *Cold start* da função pode adicionar latência à primeira autenticação — aceitável para um endpoint de auth de baixa frequência; mitigável com provisioned concurrency se necessário.
- Segredo de assinatura do JWT precisa ser compartilhado de forma segura entre a Lambda (emissor) e o gateway/aplicação (validadores) via secret manager.

Alternativas consideradas

- **Manter auth dentro do monólito:** não atende ao requisito de solução serverless nem ao ponto único de controle.
- **Ingress do Kubernetes como gateway:** cobre roteamento, mas concentra a autenticação novamente dentro do cluster e oferece menos recursos de borda que um API Gateway gerenciado.

ADR-004 — Segregação em quatro repositórios com CI/CD independente

- **Status:** Aceito
- **Data:** 2025-07
- **Decisores:** Grupo 123
- **Relacionado a:** [ADR-005](#), [ADR-003](#)

Contexto

Na Fase 2 todo o projeto (aplicação, manifestos Kubernetes, Terraform e workflows) vive em um único repositório. A Fase 3 exige **quatro repositórios separados**, cada um com CI/CD próprio e deploy automático, além de regras de proteção de branch. Componentes com ciclos de vida e permissões diferentes não devem compartilhar o mesmo pipeline e o mesmo blast radius.

Decisão

Segregar em **quatro repositórios** na organização GitHub `fiap-officine`, cada um com pipeline independente:

#	Repositório	Conteúdo	Deploy
1	<code>fiap-officine-lambda</code>	Função serverless de autenticação por CPF	Publicação da Lambda
2	<code>fiap-officine-kubernetes</code>	Terraform da infra base AWS (VPC, K3s, API Gateway, security groups)	<code>terraform apply</code>
3	<code>fiap-officine-database</code>	Terraform do banco gerenciado (RDS) + scripts SQL/migrations	<code>terraform apply</code>
4	<code>fiap-officine-api</code>	Aplicação principal (FastAPI + worker) executando no cluster	Build de imagem + rollout no K3s

Ordem de dependência de deploy: `fiap-officine-kubernetes` (VPC/K3s) → `fiap-officine-database` (RDS nas subnets privadas da VPC) → (`fiap-officine-lambda` || `fiap-officine-api`).

A VPC e as subnets privadas são provisionadas pelo repositório `fiap-officine-kubernetes`; o `fiap-officine-database` consome essas subnets para posicionar o RDS de forma isolada, acessível apenas pelos pods da aplicação e pela Lambda de autenticação.

Regras de proteção (todos os repositórios): - Branch `main` / `master` protegida, **sem commit direto**. - Merge **somente via Pull Request** (com review e checks verdes obrigatórios). - **Deploy automático** nas branches de **homologação** (`develop`) e **produção** (`main`).

Estrutura de pipeline por repositório: - *PR*: lint, testes/validação (`terraform validate`, `pytest`, `ruff`), `plan` do Terraform quando aplicável. - *Merge em develop/main*: build, publicação de artefato/imagem e deploy automático para o ambiente correspondente.

Consequências

Positivas - Cada componente evolui, versiona e é auditado isoladamente; permissões e secrets são concedidos por menor privilégio. - Falha ou rollback de um componente não arrasta os demais. - Pipelines menores e mais rápidos; histórico de mudança mais legível.

Negativas / trade-offs - Mudança que atravessa componentes exige coordenação entre repositórios (versionar contratos, respeitar ordem de deploy). - Mais configuração inicial (4 conjuntos de CI/CD, secrets e proteções). - Necessidade de documentar claramente as dependências entre repos (feito na tabela acima e no README de cada um).

Alternativas consideradas

- **Mono-repo com múltiplos pipelines (paths filter):** menos atrito de coordenação, mas não atende ao requisito

explícito de quatro repositórios e mantém permissões acopladas.

- **Separar apenas app e infra (2 repos):** insuficiente frente ao enunciado e ainda mistura banco com cluster.

ADR-005 — Banco de dados gerenciado desacoplado do cluster

- **Status:** Aceito
- **Data:** 2025-07
- **Decisores:** Grupo 123
- **Relacionado a:** [RFC-002](#), [ADR-002](#), [ADR-004](#)

Contexto

Na Fase 2 o PostgreSQL roda como um pod no próprio cluster, com `PersistentVolumeClaim`. Isso acopla o dado ao ciclo de vida do cluster, concentra a responsabilidade de backup/HA no time e não atende ao requisito de **banco de dados gerenciado** da Fase 3. A escolha do *engine* (PostgreSQL) já está justificada na [RFC-002](#); este ADR trata de **onde e como** o banco roda.

Decisão

Prover o banco como serviço **gerenciado (Amazon RDS PostgreSQL)**, **fora do cluster**, provisionado por Terraform em um **repositório dedicado** (`fiap-officine-database` , ver ADR-004).

Características: - **Multi-AZ** para alta disponibilidade e failover automático. - Instância em **subnet privada**; acesso restrito ao cluster (K3s) e à Lambda de autenticação por security group. - **TLS obrigatório** e credenciais no **Secrets Manager** (nada em manifesto). - **Migrations Alembic** aplicadas pelo pipeline do repositório de banco, antes do rollout da aplicação. - **Pool de conexões** dimensionado na aplicação para respeitar o limite de conexões da instância, considerando a escala de réplicas do HPA (db.t4g.micro em homologação, db.t4g.small em produção) (ADR-002).

Consequências

Positivas - Backup automatizado, point-in-time recovery, patching e failover a cargo do provedor — atende disponibilidade e reduz carga operacional. - Dado sobrevive à recriação do cluster; separação clara de responsabilidades entre repositórios de infra. - Escala de leitura futura via read replicas sem mudar a aplicação.

Negativas / trade-offs - Custo maior que um contêiner de banco. - Latência de rede entre cluster e banco (mitigada mantendo ambos na mesma região/VPC). - Gestão de conexões passa a ser crítica sob HPA — exige pool e, se necessário, um pooler (ex.: PgBouncer/RDS Proxy).

Alternativas consideradas

- **Postgres em contêiner no cluster (Fase 2):** não atende ao requisito de banco gerenciado e concentra risco operacional.
- **Banco serverless (Aurora Serverless):** escala fina e ótimo para carga intermitente, porém com trade-offs de custo/latência; registrado como evolução possível mantendo compatibilidade PostgreSQL.

Requests for Comments (RFCs)

RFCs documentam **decisões técnicas relevantes** — propostas colocadas em discussão antes de virarem definitivas, com motivação, alternativas avaliadas e riscos. Uma vez aceitas, alimentam os [ADRs](#) correspondentes.

RFC	Título	Status
RFC-001	Escolha do provedor de nuvem	Aceita
RFC-002	Escolha do banco de dados gerenciado	Aceita
RFC-003	Estratégia de autenticação por CPF (serverless)	Aceita
RFC-004	Estratégia de observabilidade	Aceita

Convenção de status

Rascunho → Em revisão → (Aceita | Rejeitada) → Substituída por RFC-XXX

RFC-001 — Escolha do provedor de nuvem

- **Status:** Aceita
- **Data:** 2025-07
- **Autores:** Grupo 123
- **Consolida em:** [ADR-003](#), [ADR-004](#), [ADR-005](#)

1. Resumo

Propõe-se adotar a **AWS** como provedor de nuvem para hospedar o cluster Kubernetes, o API Gateway, a função serverless de autenticação e o banco de dados gerenciado.

2. Motivação

A Fase 3 dá **livre escolha de nuvem**, mas exige um conjunto coeso de serviços: API Gateway, função serverless, banco gerenciado, cluster Kubernetes escalável e provisionamento por Terraform. A decisão precisa garantir que todas essas peças tenham serviço gerenciado maduro, boa integração entre si e com Terraform, e ferramental de observabilidade.

3. Proposta

Mapeamento dos requisitos para serviços AWS:

Requisito	Serviço AWS
API Gateway	Amazon API Gateway (HTTP API v2)
Função serverless de autenticação	AWS Lambda
Banco de dados gerenciado	Amazon RDS (PostgreSQL)
Cluster Kubernetes escalável	K3s em EC2 (free tier) + HPA
Fila de notificações	Redis 7 (in-cluster)
Segredos	AWS Secrets Manager
IaC	Terraform (provider AWS oficial e maduro)
Observabilidade	New Relic integrado (ver RFC-004)

4. Alternativas consideradas

- **Google Cloud (GKE + Cloud Functions + Apigee + Cloud SQL):** stack equivalente e forte em Kubernetes; preterido por familiaridade da equipe e amplitude do ecossistema/documentação da AWS.
- **Azure (AKS + Azure Functions + APIM + Azure Database):** igualmente capaz; mesma linha de decisão.
- **Multi-cloud:** descartado por complexidade desproporcional ao escopo.

O desenho é **portável**: cada serviço tem equivalente direto nas outras nuvens, então a decisão não amarra o domínio a AWS além da camada de infraestrutura.

5. Riscos e mitigação

- **Vendor lock-in:** mitigado por Terraform (infra reprovisionável) e por manter a aplicação em contêineres Kubernetes (portáteis) e PostgreSQL padrão.
- **Custo:** mitigado por HPA (ADR-002), banco corretamente dimensionado e serverless por uso na autenticação.

6. Decisão

AWS aprovada como provedor de referência. Consolidada nos ADRs 003, 004 e 005.

RFC-002 — Escolha do banco de dados gerenciado

- **Status:** Aceita
- **Data:** 2025-07
- **Autores:** Grupo 123
- **Consolida em:** [ADR-005](#)
- **Relacionado a:** [data-model.md](#)

1. Resumo

Propõe-se **PostgreSQL 16.9 gerenciado (Amazon RDS)** como banco de dados da aplicação.

2. Motivação

O domínio é fortemente transacional e relacional (clientes, veículos, ordens de serviço, itens, estoque, histórico). A Fase 3 exige banco **gerenciado** com consistência e performance. É preciso decidir o *engine* (relacional vs. NoSQL; qual SGBD) e formalizar a justificativa.

3. Proposta

Engine: PostgreSQL. Racional:

- **ACID e integridade referencial** — essenciais para estoque, orçamento e transições de status.
- **Modelo relacional rico** — consultas de negócio dependem de JOINS e chaves estrangeiras (OS por cliente, tempo médio por status, histórico).
- **Tipos adequados** — `NUMERIC` para dinheiro, `ENUM` nativo para status, `TIMESTAMPTZ` para datas com fuso.
- **Ecossistema já adotado** — SQLAlchemy 2.0 + Alembic, sem custo de migração de tecnologia.

Modalidade: gerenciada (RDS), Multi-AZ, com backup automático, TLS e credenciais em Secrets Manager. Detalhes operacionais na ADR-005.

4. Alternativas consideradas

Opção	Por que não
MySQL gerenciado	Capaz, mas o projeto já usa recursos PostgreSQL (ENUM nativo, tipos) e o time tem mais domínio
SQL Server	Custo de licenciamento sem benefício para o caso de uso
NoSQL (DynamoDB/MongoDB)	Modelo de acesso é relacional; reconstruir integridade/JOINS em NoSQL adicionaria complexidade sem ganho
Aurora PostgreSQL	Compatível e escalável; mantido como evolução futura por custo/simplicidade nesta fase

5. Riscos e mitigação

- **Conexões sob HPA:** réplicas em escala podem esgotar conexões — mitigado com pool na aplicação e, se necessário, RDS Proxy/PgBouncer (ADR-005).
- **Latência cluster↔banco:** mantê-los na mesma região/VPC.

6. Decisão

PostgreSQL 16.9 em Amazon RDS aprovado. Ajustes de modelo (índices, consistência de estoque) descritos em [data-model.md](#).

RFC-003 — Estratégia de autenticação por CPF (serverless)

- **Status:** Aceita
- **Data:** 2025-07
- **Autores:** Grupo 123
- **Consolida em:** [ADR-003](#)

1. Resumo

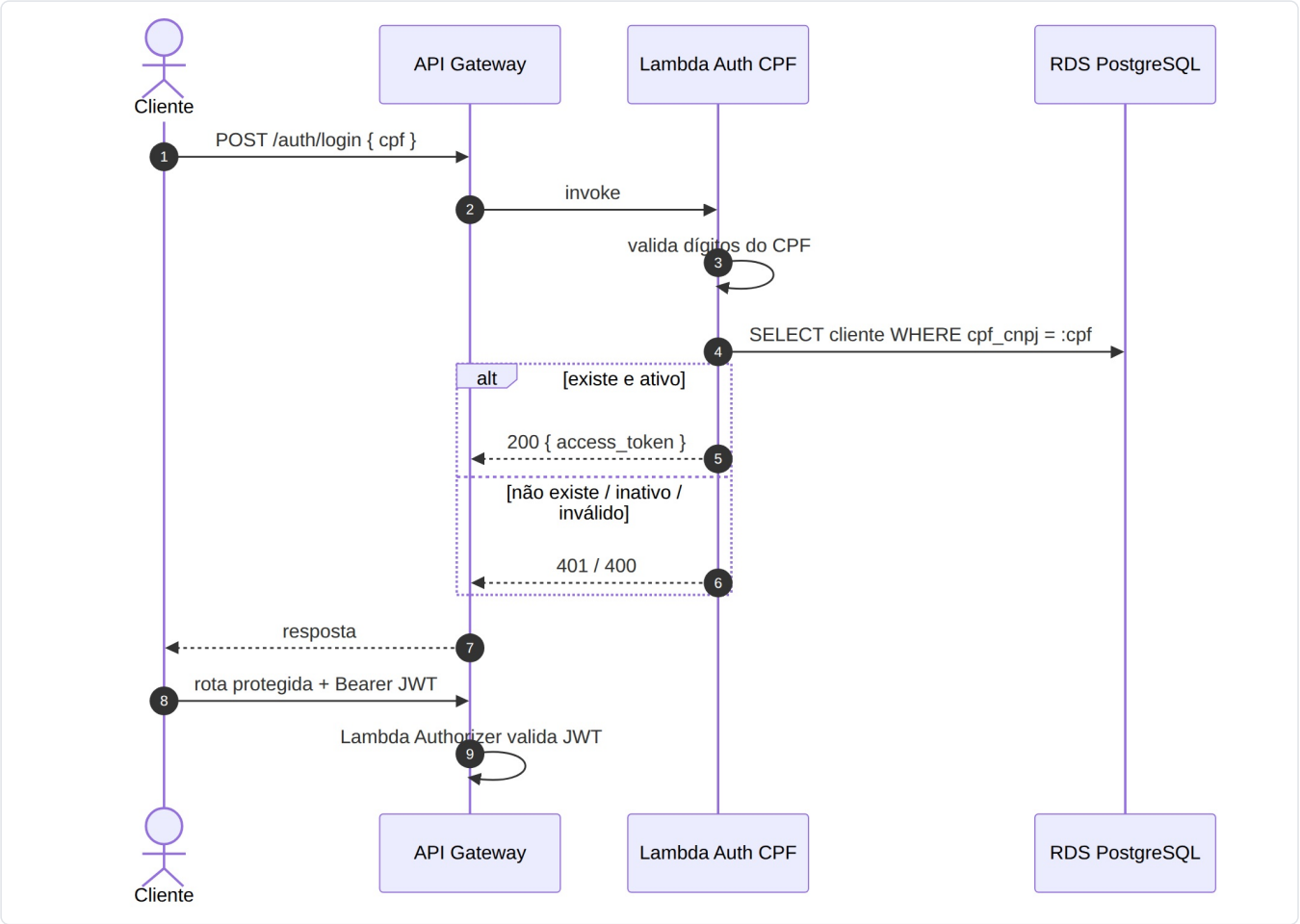
Propõe-se autenticar o cliente por **CPF** através de uma **função serverless (AWS Lambda)** exposta pelo **API Gateway**, que valida o documento, confirma a existência/status do cliente no banco e devolve um **JWT** para consumo das rotas protegidas.

2. Motivação

A Fase 3 exige: proteger rotas sensíveis com autenticação via CPF, uma **função serverless** que valide o CPF, consulte a base e gere um JWT. Na Fase 2 a autenticação (usuário/senha) estava embutida no monólito — não é serverless e acopla auth à aplicação.

3. Proposta

3.1 Fluxo



3.2 Contrato

- **Entrada:** **POST /auth/login** com `{ "cpf": "12345678909" }`.

- **Saída (sucesso):** `{ "access_token": "<jwt>", "token_type": "bearer" }`.
- **Erros:** `400` CPF inválido; `401` cliente inexistente ou inativo.

3.3 Token

- JWT assinado (HS256 ou RS256), claims mínimos: `sub` (id/cpf do cliente), `exp`, `iat`. Expiração curta e configurável.
- Chave de assinatura no **Secrets Manager**, compartilhada com o validador do gateway.
- Reaproveita a validação de CPF já existente no domínio (`app/domain/validators.py`), evitando divergência de regra.

3.4 Fronteira de responsabilidade

- **Cliente (CPF):** autenticado pela Lambda serverless — foco do requisito.
- **Operador/Admin (usuário/senha):** permanece com fluxo próprio da aplicação, fora do escopo desta RFC.

4. Alternativas consideradas

- **Cognito / provedor de identidade gerenciado:** robusto, porém excessivo para um login por CPF sem senha e menos aderente ao "escreva a função que valida o CPF" pedido no enunciado.
- **Auth no Ingress do cluster:** volta a concentrar autenticação dentro do cluster; não é serverless.

5. Riscos e mitigação

- **Cold start** na primeira chamada — aceitável para auth; provisioned concurrency se necessário.
- **CPF como identificador sensível** — não logar o CPF em claro; mascarar em logs (RFC-004) e trafegar sempre sobre TLS.
- **Acesso da Lambda ao banco** — exige a função na VPC com security group restrito ao RDS.

6. Decisão

Aprovada. Autenticação de cliente por CPF via Lambda + API Gateway, emitindo JWT. Consolidada na ADR-003.

RFC-004 — Estratégia de observabilidade

- **Status:** Aceita
- **Data:** 2025-07
- **Autores:** Grupo 123
- **Relacionado a:** [ADR-001](#), [ADR-002](#)

1. Resumo

Propõe-se adotar o **New Relic** como plataforma de observabilidade, cobrindo logs estruturados, métricas, APM (traces distribuídos) e alertas, com dashboards de negócio e de infraestrutura construídos em **NRQL**.

2. Motivação

A fase exige visibilidade total: monitorar latência das APIs, consumo de recursos do Kubernetes, healthchecks/uptime, alertas de falha no processamento de OS e logs estruturados com correlação entre requisições — além de dashboards de negócio.

3. Proposta

3.1 Coleta

- **New Relic Python Agent (APM)** instrumentando a aplicação FastAPI — transações, traces distribuídos e custom events, com propagação de contexto do API Gateway ao worker.
- **New Relic Infrastructure Agent** como `DaemonSet` (`newrelic-infrastructure`) no cluster — coleta métricas de CPU/memória por pod e por nó (`K8sContainerSample` / `K8sNodeSample`), reinícios de pod, etc.
- **New Relic Synthetics** para monitor de uptime externo, checando `/health` em intervalo curto.
- **Métricas do API Gateway** (latência de borda, 4xx/5xx) integradas à conta.

3.2 Logs estruturados (JSON)

- Um `ObservabilityMiddleware` na aplicação injeta **correlation ID** e mede latência por requisição; um `JSONLogFormatter` emite todo log em **JSON**.
- Campos mínimos: `timestamp`, `level`, `service`, `trace_id`, `correlation_id`, `path`, `status_code`, `latency_ms`.
- **Correlação:** o `correlation_id` acompanha a requisição pela API e pelo evento publicado na fila, permitindo rastrear uma OS de ponta a ponta (API -> Redis -> worker).
- Os logs vão para `stdout` e são encaminhados ao **New Relic Log Management**, já correlacionados com as transações do APM.
- **Dados sensíveis:** CPF e credenciais nunca em claro — mascarados no logger.

3.3 Monitores e alertas

Sinal	Condição de alerta
Latência da API	p95 acima do limite por X min
Erros	taxa de 5xx acima do limite
Recursos K8s	CPU/memória saturando (correlaciona com HPA)
Healthcheck/uptime	<code>/health</code> falhando (Synthetics)
Falha no processamento de OS	erro/consumo travado no worker da fila

3.4 Dashboards (NRQL)

- **Negócio:** volume diário de OS; tempo médio de execução por status (Diagnóstico, Execução, Finalização); erros/falhas nas integrações.
- **Infra:** latência (p50/p95/p99) e throughput por rota; CPU/memória por pod; réplicas ativas (HPA); saúde do banco e da fila.

3.5 Healthchecks

- Endpoint `/health` (e `/api/v1/health`) expõe estado da API, do PostgreSQL e do Redis; usado por readiness/liveness probes no Kubernetes e pelo monitor de uptime do New Relic. Rotas de saúde e documentação (`/docs`, `/openapi.json`, `/redoc`) ficam abertas (`authorization_type = NONE`) no API Gateway.

4. Alternativas consideradas

- **Datadog:** equivalente e também aceito pelo enunciado; foi a primeira opção avaliada. **New Relic foi escolhido** pela cobertura do free tier atender a homologação, pela experiência com NRQL e pela integração nativa do agente Kubernetes. A decisão é reversível — logs em JSON + OpenTelemetry mantêm a portabilidade da instrumentação.
- **Stack self-hosted (Prometheus + Grafana + Loki):** poderoso, mas adiciona carga operacional significativa frente a uma solução gerenciada.

5. Riscos e mitigação

- **Custo por volume de logs/traces:** mitigado com amostragem de traces, níveis de log por ambiente e uso do free tier em homologação.
- **Vazamento de dado sensível em log:** mitigado por mascaramento no logger e revisão dos campos emitidos.

6. Decisão

New Relic aprovado, com logs JSON correlacionados, APM/traces distribuídos, monitores de latência/erro/recurso/uptime/falha de OS e dashboards NRQL de negócio e infraestrutura.